Note méthodologique : preuve de concept

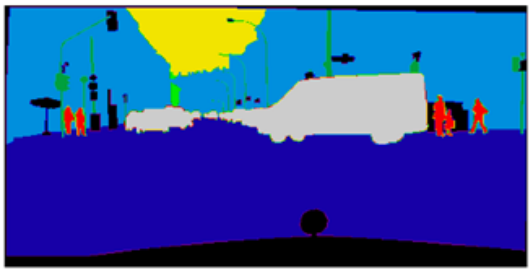
Dataset retenu

Le dataset retenu est celui du jeu de données « Cityscapes » du P8 : « Participez à la conception d'une voiture autonome »

Le jeu de données est composé de quelques milliers d'images de routes urbaines allemandes, prise depuis l'habitacle d'une voiture, ainsi que, pour chacune d'elles, des fichiers d'annotation correspondants dont les images masques dont nous avons besoin pour la segmentation sémantique.



Image réelles 2048*1024

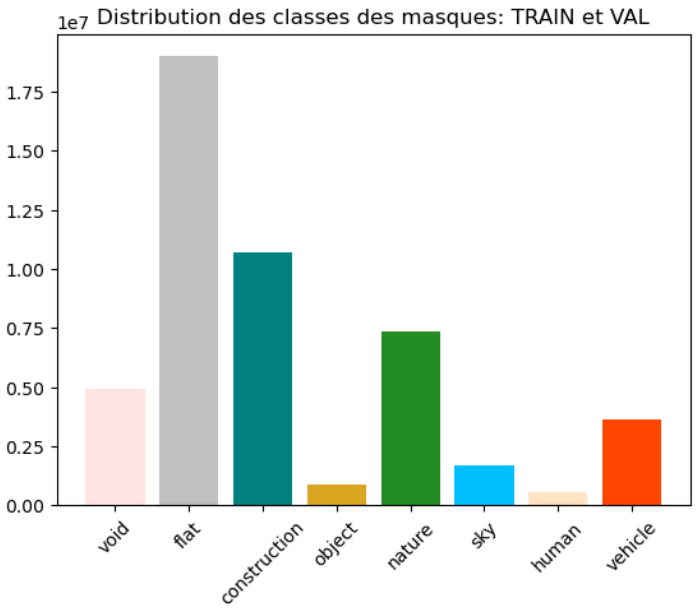


(Ground Truth sur 30 labels)

Nous allons utiliser 2975 images et masques pour **l'entrainement** et 500 images et masques pour le jeu de **test**.

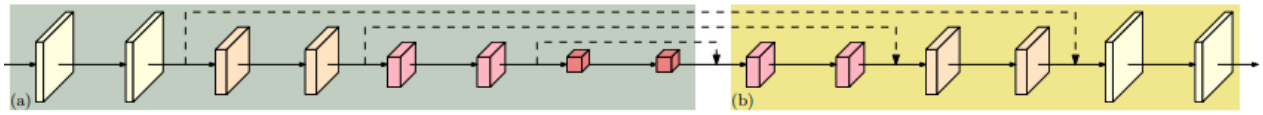
A l'origine la taille des images et masques est de 2048 * 1024 mais pour des questions de performances et de faisabilité nous allons retailler les images et maques en 128 * 128.

Pour les mêmes raisons mais aussi pour plus de simplicité, les classes de pixels qui s'élèvent initialement à 30 vont être regrouper en 8 labels dont la répartition est la suivante :



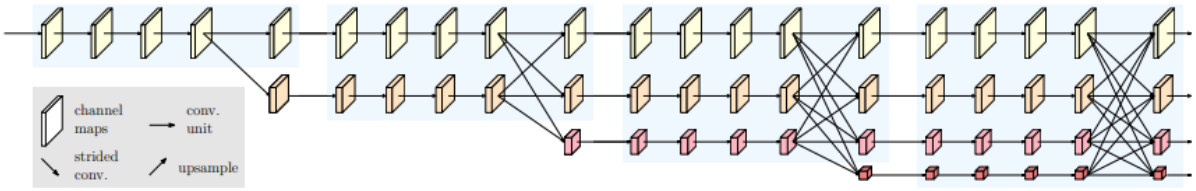
Les concepts de l'algorithme récent

Le nouvel Algorithme que nous avons choisi pour ce POC est l'**HRNet**(v2) (**H**igh **R**esolution **Net**work) qui est à l'instar de **UNet** un réseau de neurones de type **CNN**.



UNet CNN

Mais contrairement à **UNet** qui est constitué d'une architecture en **série** de type Encodeur – Décodeur, **HRNet** suit une différente approche dont voici une représentation graphique et les principales caractéristiques :



HRNet CNN

<u>1/ Le maintien de la haute résolution</u>

Le modèle **HRNet** maintient des flux de convolutions à haute résolution via son réseau principal (le plus en haut sur la figure) dont l'output sera exploité en fin de réseau.

<u>2/ Les réseaux parallèles</u>

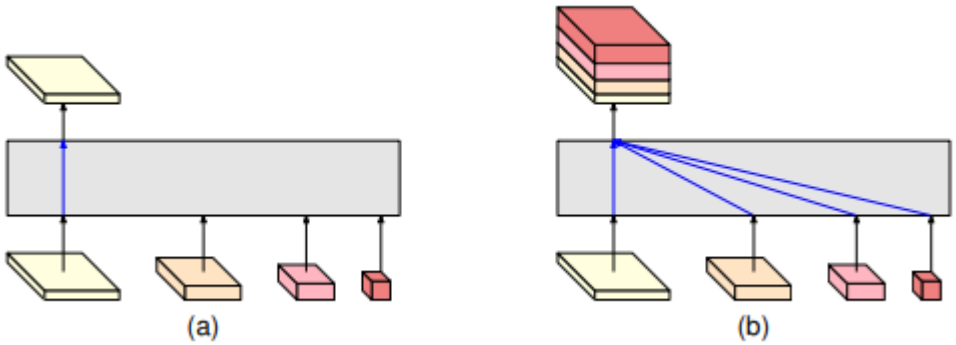
En ce qui concerne les réseaux parallèles, **HRNet** utilise plusieurs flux de convolutions de résolutions différentes, qui sont connectés en parallèle. Ces flux parallèles sont construits progressivement, en ajoutant des flux de résolutions plus basses à chaque étape. Ainsi, les flux parallèles d'une étape ultérieure comprennent les résolutions des étapes précédentes ainsi qu'une résolution supplémentaire plus basse.

<u>3/ La fusion des sous réseaux</u>

La fusion des sous-réseaux dans **HRNet** se fait en échangeant de manière répétée les informations entre les résolutions. Cette fusion multi-résolution permet de combiner les informations provenant des différents flux parallèles et d'obtenir une représentation finale qui est à la fois sémantiquement riche et spatialement précise.

Ces caractéristiques du modèle **HRNet**, telles que le maintien de la haute résolution, l'utilisation de réseaux parallèles et la fusion des sous-réseaux, permettent de conserver des informations plus riches et une précision spatiale plus élevée dans des tâches telles que la segmentation sémantique et la détection d'objets.

Aussi pour notre POC nous avons choisi la **version 2** de **HRNet** qui introduit principalement une meilleure fusion d'information entre les sous-réseaux comme illustré ci-dessous :



(a) HRNetV1 : ne produit que la représentation du flux de convolution à haute résolution.
(b) HRNetV2 : Concaténation des représentations (suréchantillonnées)

La version 2 du modèle conduit alors à une meilleure précision et à une meilleure représentation des détails de l'image.
Aussi le modèle utiliser comprend de l'OCR (Object-Contextual Representation) qui utilise une représentation contextuelle des objets pour caractériser chaque pixel en exploitant la représentation de la classe d'objet correspondante, mais comme nous pouvons le voir son apport n'est pas décisif si l'on regarde le score mIoU des 2 lignes du bas du tableau ci-dessous produit par les concepteurs **d'HRNet** :

	backbone	#param.	GFLOPs	mIoU
UNet++ [189]	ResNet-101	59.5M	748.5	75.5
Dilated-ResNet [54]	D-ResNet-101	52.1M	1661.6	75.7
DeepLabv3 [20]	D-ResNet-101	58.0M	1778.7	78.5
DeepLabv3+ [22]	D-Xception-71	43.5M	1444.6	79.6
PSPNet [181]	D-ResNet-101	65.9M	2017.6	79.7
HRNetV2	HRNetV2-W40	45.2M	493.2	80.2
HRNetV2	HRNetV2-W48	65.9M	696.2	81.1
HRNetV2 + OCR [170]	HRNetV2-W48	70.3M	1206.3	**81.6**

La modélisation

1/ Choix des modèles

Pour le modèle baseline nous avons pris un Réseau **UNet** avec backbone **mobilenet** qui est celui qui nous avait donné les meilleurs résultats lors du P8.

Concernant le modèle **HRNet**, nous n'avons pas eu tellement le choix.

Après d'innombrables recherches et tests de code nous avons finalement réussi à faire fonctionner le modèle HRNet(v2) +OCR disponible sur la page github suivante :
https://github.com/JanMarcelKezmann/TensorFlow-Advanced-Segmentation-Models
Cette librairie nous a permis également de réimplémenter UNet + Mobilnet afin de pouvoir comparer les 2 algorithmes (baseline + new) depuis la même librairie.

2/ Optimisations

Pour optimiser les résultats nous avons prix la taille maximale d'image qui ne conduisait pas un crash de notre nouveau modèle pour cause de limitation de ressources, à savoir une taille de **128*128**.
Pour les mêmes raisons nous avons dû limiter la taille du batch de notre dataloader à **4**.

Afin d'éviter le surentrainement tout en économisant de la ressource machine, nous avons mis en place un **early stopp**ing avec une **patience** à **5** sur la « **val_loss** » qui fait que l'entrainement de notre modèle s'arrêtera prématurément si pendants **5** epochs consécutifs il n'y a pas d'amélioration de la **val_loss** : **5** étant la valeur de patience trouvée après plusieurs itérations et qui donnait des résultats de métrique à peu près convenables.

Les poids d'entrainements des modèles ont été également sauvegardés dans un fichier de type **.h5** et sont récupérés si le fichier existe lors de la réexécution (assez rapide) du notebook.

Nous avons également choisi délibérément de ne pas utiliser l'**augmentation** de données car il a déjà été vu lors du P8 que pour le jeu de données « **Cityscapes** » dont les images sont déjà **pré-nettoyées**, l'apport de l'augmentation de données n'est pas remarquable voir un peu trop négatif quant aux valeurs de métriques obtenues.

2/ Choix des métriques

Nous en avons choisi 2 qui sont :

- **Categorical_Accuracy** : qui n'est autre que le pourcentage de pixels correctement prédit
- Mean Intersection over Union (**mIoU**) : la **moyenne des rapports** entre l'intersection de la **zone de prédiction** et la **zone réelle** (pour une classe de pixel) **SUR l'union** de ces 2 mêmes zones. (la zone pour la classe de pixels prédite se trouvant sur le masque prédit par le modèle et la zone réelle pour cette même classe se trouvant sur le masque GT (Ground Truth)

Pour simplifier la compréhension de métrique **IoU** : si la zone de segmentation prédite chevauche parfaitement celle de la réalité terrain nous auront un score de 1 et 0 dans le cas opposé.

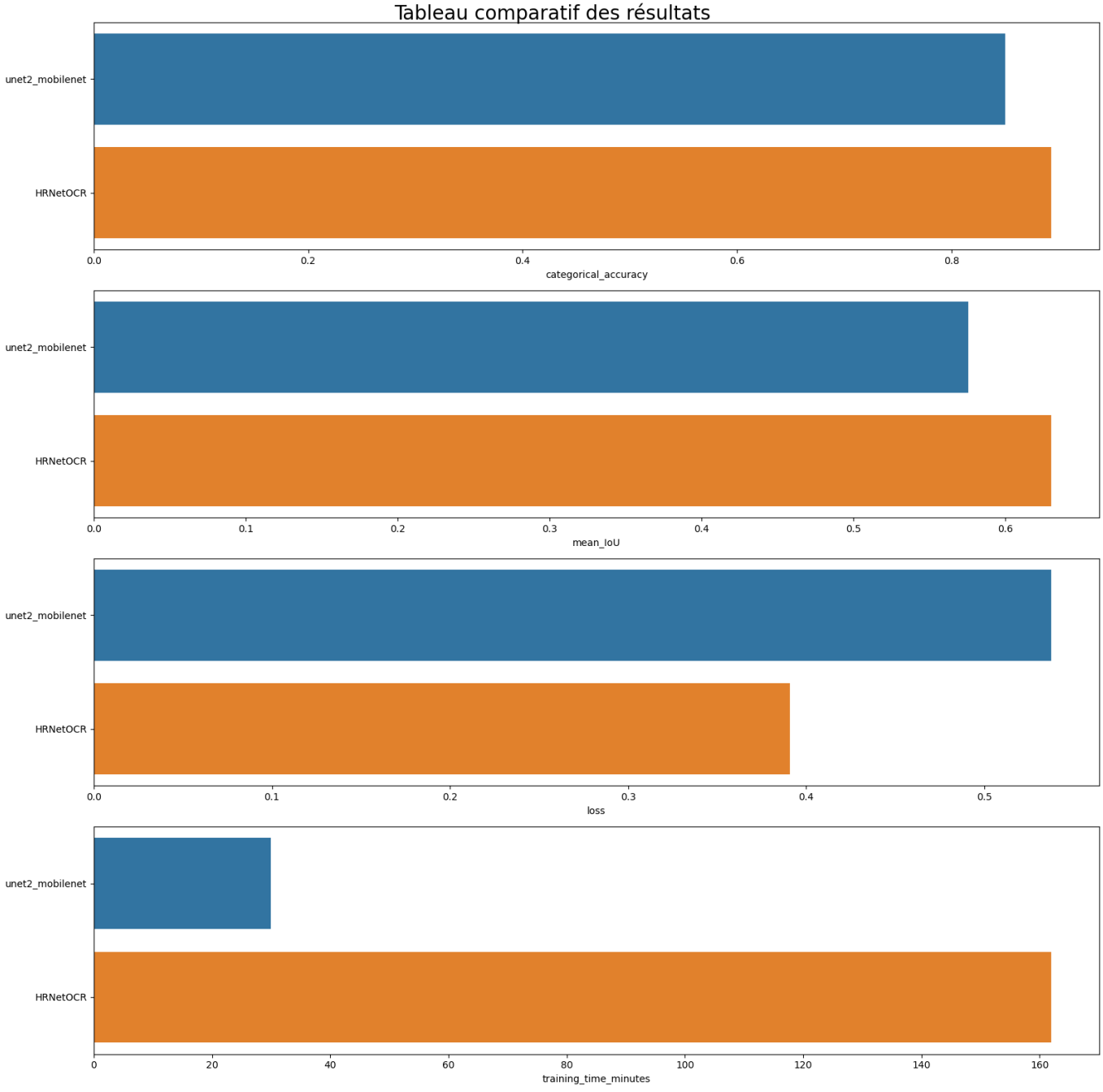
Cette métrique est importante pour palier au fait qu'il y aurait une classe de pixel majoritairement représentée ce qui pourrait conduire à score de Categorical Accuracy très élevée pour un modèle qui n'aurait pu prédire que cette classe de pixel à chaque fois !

Une synthèse des résultats

Voici les résultats obtenus sous forme de tableau et graphes :

	training_time_minutes	categorical_accuracy	mean_IoU	loss	nb_epochs
unet2_mobilenet	30	0.849978	0.575731	0.537702	16
HRNetOCR	162	0.893256	0.630250	0.390830	22

Tableau comparatif des résultats



Nous pouvons conclure que le modèle **HRNetV2** présente de meilleurs résultats de métriques que le modèle **UNet** que ce soit pour la **Categorical Accuracy** ou le **meanIoU**, son défaut étant le temps d'entrainement qui est bien plus long (pas uniquement dû au nombre d'epochs plus élevé mais aussi car chaque epoch d'apprentissage durait beaucoup plus longtemps pour l'HRNet que le UNet)

Les limites et les améliorations possibles

1/ Limites

- Contrairement au modèle baseline UNet le modèle HRNet s'est avéré très gourmand en ressources machines lors de la phase d'apprentissage et même lors de la phase d'évaluation du jeu de test, cela s'est vu en temps d'apprentissage et entendu avec le bruit émis par notre GPU : RTX 4070.
- La taille des images et de masques ainsi que celle du batch du Dataloader a dû être revu à la baisse plusieurs fois en dépit des 12Go de VRAM du GPU, ce qui fait que les résultats obtenus sont moins bons que lors du P8.
- (La bibliothèque utilisée semble également produire de moins bons résultats, en effet le modèle UNet + mobilenet du P8 présentait de meilleurs résultats de métriques par rapport au modèle Unet + mobilenet du ce POC issue de la nouvelle librairie.)

2/ Améliorations envisageables

- Il faudrait pouvoir utiliser un GPU bien plus véloce en nombre du tensor cores et qui possèdent également plus de VRAM, (ex RTX 3090/4090 ou un GPU PRO Nvidia)
- Dans l'idéal, il aura fallu pouvoir trouver une librairie dont les résultats de métrique du modèle UNet (+ backbone mobilenet) se rapprochent de ceux du modèle UNet du projet 8.